

Experiment:- 14

Problem statement:-

Linux provides powerful utilities to monitor user activity, manage files, and control process execution priorities. System administrators and cyber security professionals often need to determine how many users are currently logged in, list files in structured formats, and manage jobs running in foreground or background with controlled priorities.

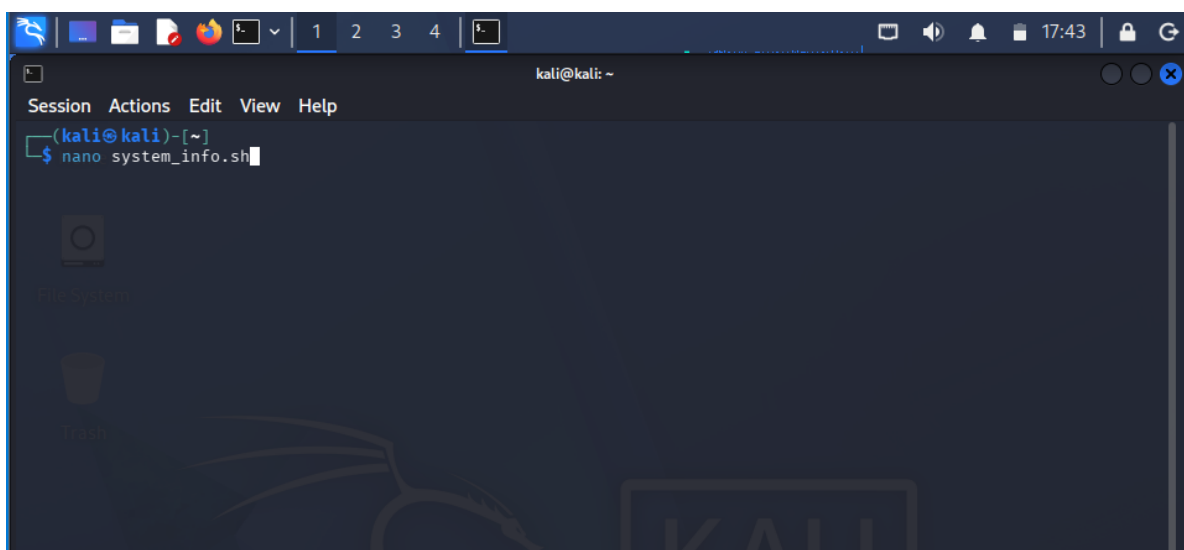
The problem is to write shell scripts that display the number of users logged into the system, list files in a columnar format, and manage job execution using priority and background execution, thereby demonstrating system monitoring and job control through Bash scripting.

Aim:- Shell script for user login count, file listing and job control.

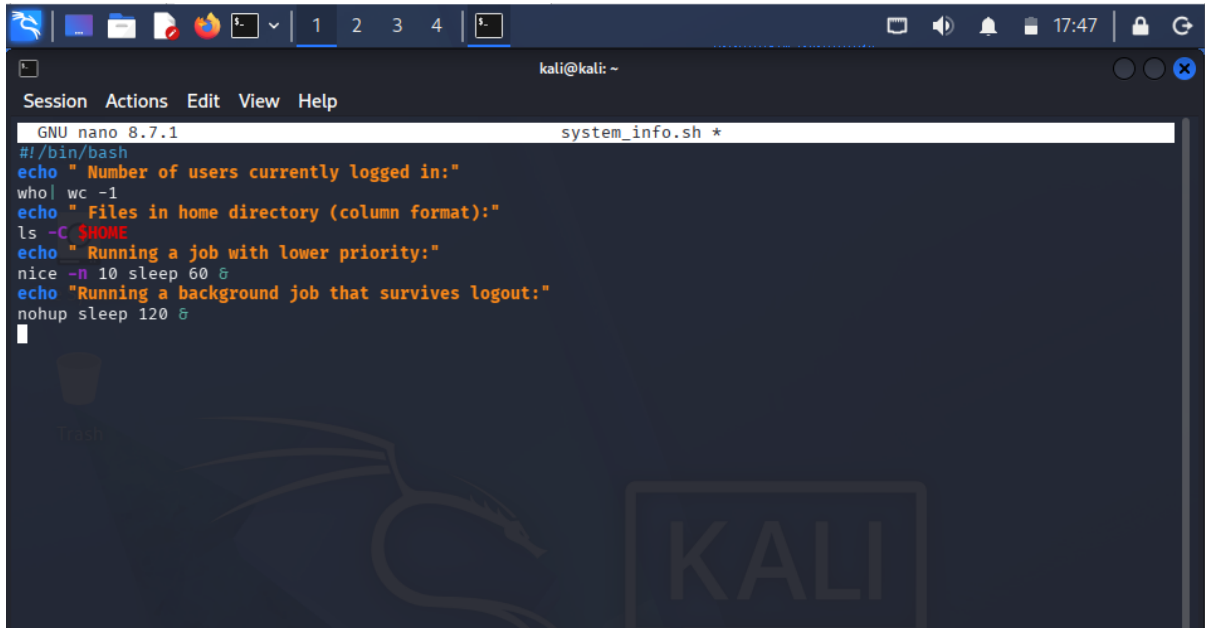
Procedure:-

A. Writing the Shell Script

1. Open the terminal.
2. Create a script file: nano system_info.sh



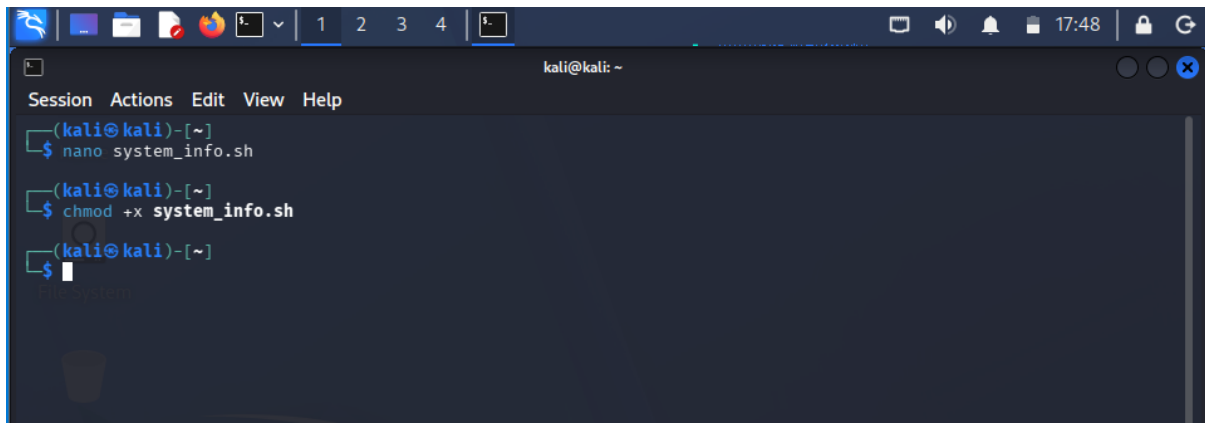
3. Enter the script:



```
GNU nano 8.7.1 system_info.sh *
#!/bin/bash
echo " Number of users currently logged in:"
who| wc -l
echo " Files in home directory (column format):"
ls -C $HOME
echo " Running a job with lower priority:"
nice -n 10 sleep 60 &
echo "Running a background job that survives logout:"
nohup sleep 120 &
```

4. Save and exit the editor.

5. Make the script executable: `chmod +x system_info.sh`



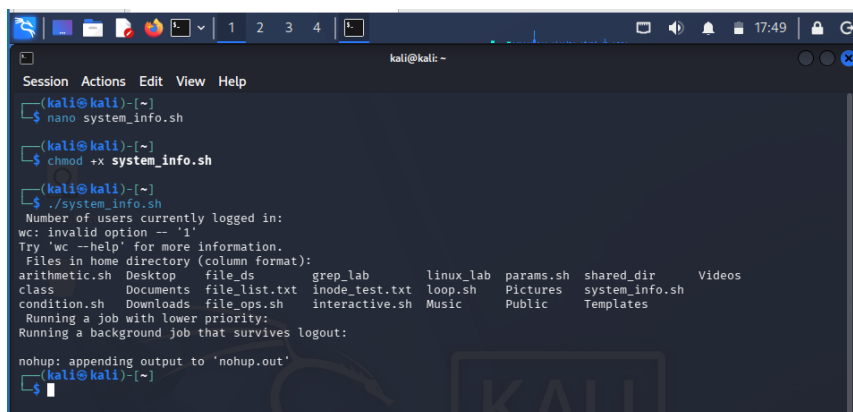
```
(kali@kali)-[~]
└─$ nano system_info.sh

(kali@kali)-[~]
└─$ chmod +x system_info.sh

(kali@kali)-[~]
└─$
```

Executing the Script

6. Run the script: `./system_info.sh`



```
(kali@kali)-[~]
└─$ nano system_info.sh

(kali@kali)-[~]
└─$ chmod +x system_info.sh

(kali@kali)-[~]
└─$ ./system_info.sh
Number of users currently logged in:
wc: invalid option -- 'l'
Try 'wc --help' for more information.
Files in home directory (column format):
arithmetic.sh Desktop file_ds grep_lab linux_lab params.sh shared_dir Videos
class Downloads file_list.txt inode_test.txt loop.sh Pictures system_info.sh
condition.sh Downloads file_ops.sh interactive.sh Music Public Templates
Running a job with lower priority:
Running a background job that survives logout:
nohup: appending output to 'nohup.out'

(kali@kali)-[~]
└─$
```

